Non-Graded Project: Optimize & Ship Your LLM App v1.0

Description

You are the Lead AI Engineer for a startup preparing to launch its first LLM-powered product. The prototype works, but latency is inconsistent, costs are rising, and leadership is unsure the system is production-ready.

In this project, you will take the app from “it works” to “it ships.” Using the provided evaluation harness, you will:

- Benchmark the current system to create a reproducible baseline
- Optimize prompts for stability and token efficiency
- Implement a resilient middleware layer for API calls
- Run an A/B/C experiment comparing latency, cost per task, and quality

  You will conclude with a concise Ship/No-Ship memo and an optional canary rollout plan.

Learning Objectives

By the end of this project, learners will be able to:

- Run a reproducible baseline using a lightweight evaluation harness
- Optimize prompts for stability and cost efficiency
- Implement middleware that adds retries, backoff, caching, and token logging
- Conduct an A/B/C experiment across latency, throughput, cost, and win-rate
- Recommend a Ship/No-Ship decision with rollout and rollback criteria

Introductory Slide

Title: From Prototype to Production: Optimize & Ship Your LLM App v1.0

Subtitle: Turn “it works” into “it ships” with benchmarks, middleware, and data-driven decisions.

Tools to Be Used:

- Python 3.10+
- OpenAI API (or compatible LLM endpoint)
- pandas (data analysis)
- matplotlib (visualizations)
- JSON / CSV datasets
- Git
- Optional: Terminal / CLI, VS Code

Scenario

Your startup is days away from announcing its new LLM-powered assistant. Under real traffic, latency spikes, costs rise unpredictably, and stakeholders disagree on whether the system is launch-ready.

You are the Lead AI Engineer and must determine whether the current configuration is safe to ship. You are provided with:

- A benchmark harness that exports JSONL logs and CSV summaries
- A basic middleware wrapper for LLM API calls

- A starter configuration for Variants A (baseline), B (faster/cheaper), and C (quality-oriented)

Your job is to:

1. Establish a clean baseline
2. Improve prompts and middleware
3. Run an A/B/C comparison
4. Deliver a Ship/No-Ship recommendation

Instructions for Learners

Step 1: Establish the Baseline

1. Clone or download the project repository.
2. Review the evaluation harness and dataset.
3. Run a baseline evaluation for Variant A
   (e.g., python run_eval.py --variant A).
4. Confirm that JSONL logs and an analysis-ready CSV are generated.
5. Review p50/p95 latency, tokens/sec, and cost per task.
6. Note at least two performance concerns.

Outcome: A reproducible baseline and initial observations.

Step 2: Optimize Prompts for Stability & Cost

1. Open the prompt templates.
2. Identify ways to reduce verbosity and tighten structure:
   - JSON formatting
   - Fewer decorative phrases
   - Self-checks (schema or reasoning checks)

3. Create an updated prompt pack (e.g., prompts_v2.md).
4. Re-run a small subset with Variant A to confirm:
   - Quality is stable or improved
   - Cost is stable or reduced
   - p95 latency does not regress

Outcome: An optimized prompt pack and a short note (2–3 sentences) explaining how it reduces variance or cost.

Step 3: Implement / Refine Resilient Middleware

1. Open the middleware code that wraps LLM API calls.
2. Ensure it supports:
   - Rate limit handling with jittered backoff
   - Timeouts for TTFT and total latency
   - Basic caching for deterministic prompts
   - Logging of tokens, latency, cost, retries, and cache hits

3 Run a small test script to verify logs and stability.

Outcome: A middleware layer that handles mild failures and generates useful telemetry.

Step 4: Design and Run an A/B/C Experiment

Define three variants:

- A: baseline prompts + baseline middleware
- B: optimized prompts + conservative middleware improvements
- C: optimized prompts + more advanced settings or model tier

Run the harness on the same dataset for all variants with identical parameters. Your CSV should contain:

- p50/p95 latency
- tokens/sec
- cost per task
- win-rate vs Variant A

Create one chart (matplotlib or similar) comparing variants.

Outcome: A single CSV and chart illustrating clear trade-offs.

Step 5: Write Your Decision Memo & Optional Canary Plan

Write a 250-word memo that includes:

- Which variant to ship and why
- Evidence from latency, cost, and quality metrics
- A rollback trigger (e.g., “rollback if p95 worsens by 20% for 10 minutes”)

(Optional) Add a brief canary rollout plan with exposure stages and SLO gates.

Outcome: A decision memo and rollout plan leadership can act on.

Requirements for Submission (AI-Graded)

Requirement 1: Experiment Report (Baseline + A/B/C Results)

Prompt:
“Upload your experiment report containing the baseline and A/B/C evaluation results, including p50/p95 latency, tokens/sec, cost per task, and win-rate vs baseline.”

Types: .csv, .json, .ipynb, .py, .pdf, .png, .jpg, .md, .html

Requirement 2: Middleware Script (Resilient LLM Client)

Prompt:
“Upload your middleware implementation that handles rate limits, retries with backoff, basic caching, and token logging.”

Types: .py, .js, .ts, .json, .yml, .md, .txt, .pdf, .mp4, .mov

Requirement 3: Optimized Prompt Pack

Prompt:
“Upload your optimized prompt(s), including structured reasoning and self-checks, plus a short note explaining how they reduce variance and/or cost per task.”

Types: .txt, .md, .pdf, .docx, .mp3, .wav, .mp4, .mov, .webm

Requirement 4: Ship/No-Ship Decision Memo & Optional Canary Plan

Prompt:
“Provide your 250-word Ship/No-Ship decision memo, including rollback triggers and, optionally, a brief canary rollout plan with success metrics and fallback criteria.”

Types: .txt, .md, .pdf, .docx, .mp3, .wav, .aac, .mp4, .mov, .webm

Summary

This activity provides hands-on experience with evaluating LLM configurations through benchmarking, prompt optimization, and resilient API middleware. Participants gain competency in running structured experiments, comparing variants, and writing data-driven deployment recommendations—skills directly applicable to production-grade AI systems.

Resources

Primary Materials:

- Evaluation harness (run_eval.py)
- Starter dataset (data/test_cases.jsonl)
- Baseline prompt pack (prompts/)
- Middleware template (client/middleware.py)
- Variant configuration (config/variants.yaml)

Reference Documentation:

- OpenAI API Docs
- pandas and matplotlib documentation

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.